Ejercicio 04 · Unidad 02 (U02.LAB07)

Clasificador visual con CNN y Model Card sobre Fashion-MNIST

Estudiante	Edwing Perez
Repositorio	https://github.com/edvvdev/unidad02-02
Curso	INF-8239 Ciencia de Datos II · Unidad 02
Laboratorio	U02.LAB07 · Vision computacional: CNN reproducible con Fashion-MNIST
Fecha del informe	2026-11-05

Entrega academica: auditoria del dataset y particion reproducible, baseline denso frente a CNN con la misma particion y semilla, metricas globales y por clase, matriz de confusion, analisis de errores por clase, comparacion de costo y tamano, decision tecnica, notebook ejecutado, pruebas y Model Card.

Reproducibilidad. Todo el contenido de este informe se genera con los comandos del README: `uv sync --extra cpu` seguido de `bash scripts/run_all.sh`, que deja los artefactos en `reports/` y regenera este PDF. Ninguna cifra esta escrita a mano: todas se leen de `reports/cv_metrics.json`, `reports/cv_per_class.csv`, `reports/cv_errors_by_class.csv` y `reports/dataset_audit.json`.

1. Identificacion del dataset y particion

Dataset. Fashion-MNIST, benchmark publico de moda en escala de grises. Imagenes de 28x28 pixeles, canal unico, 10 clases. Procedencia: descarga automatica de `tf.keras.datasets.fashion_mnist` (ficha de Keras, licencia MIT). El bloque oficial de entrenamiento tiene 60000 imagenes y el de prueba 10000.

Preprocesamiento. Conversion a float32, division por 255 (rango [0, 1]) y anadido del canal, de modo que la forma de entrada es (N, 28, 28, 1). Rango comprobado en la auditoria: [0, 1].

Particion reproducible. Fija y determinista: las ultimas 6000 imagenes del bloque de 60.000 son validacion y las 54000 restantes son entrenamiento. Las 10000 imagenes del bloque oficial de prueba no se usan nunca durante el entrenamiento. Semilla **42** en Python, NumPy y TensorFlow.

Particion	Imagenes	Forma	Rango	Desbalance max/min	Reparto min-max por clase
Entrenamiento	54000	(28, 28, 1)	[0, 1]	1.0145	5367 - 5445
Validacion	6000	(28, 28, 1)	[0, 1]	1.1405	555 - 633
Prueba	10000	(28, 28, 1)	[0, 1]	1.0000	1000 - 1000

El reparto por clase es casi uniforme (cada clase representa cerca del 10 % en las tres particiones), de modo que el promedio global no esta inflado por clases mayoritarias.

2. Resultados: baseline denso frente a CNN

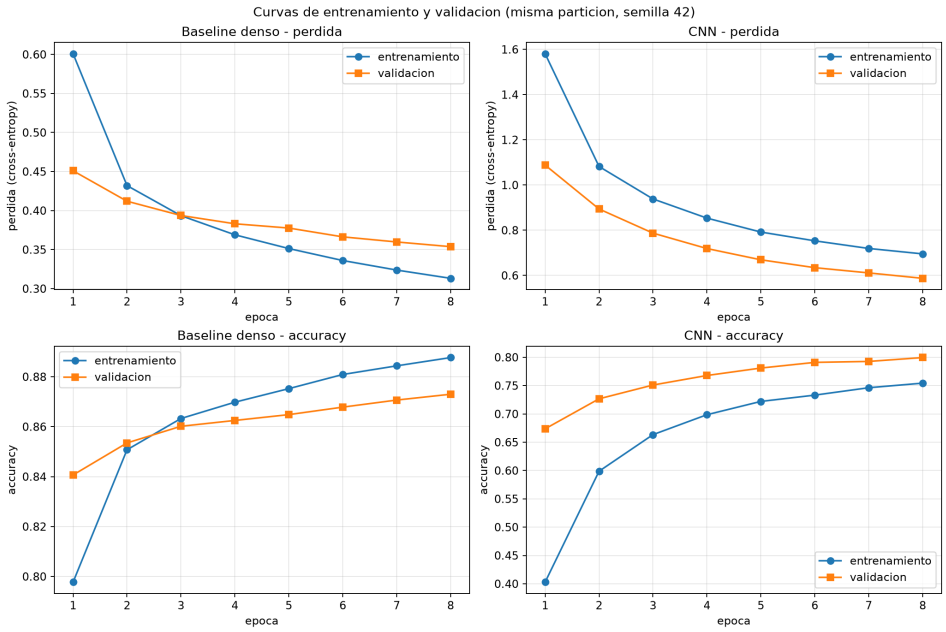
Los dos modelos se entrenan con la **misma particion** (54.000 / 6.000 / 10.000) y la **misma semilla (42)**, con optimizador Adam, batch 128 y EarlyStopping (patience 2, restore_best_weights=True), con un maximo de 8 epocas. El tiempo de inferencia es la **mediana de tres mediciones** sobre las 10.000 imagenes de prueba.

Metrica	Baseline denso	CNN	Diferencia (CNN - denso)
F1 macro	0.8647	0.7892	-0.0755
Accuracy	0.8663	0.7930	-0.0733
Parametros entrenables	50890	19466	-31424
Tamano del modelo (.keras)	619.12 KB	262.17 KB	-356.95 KB
Tiempo de entrenamiento	12.4 s	62.4 s	+50.0 s
Inferencia (mediana de 3)	0.0379 ms/img	0.0899 ms/img	+0.0520 ms/img
Epocas ejecutadas	8	8	-

Curvas. El baseline denso converge con suavidad: la perdida de entrenamiento baja de 0.6009 a 0.3132 y la de validacion de 0.4510 a 0.3538; ambos indicadores mueven en la misma direccion, el modelo esta aprendiendo patrones utiles y no hay sobreajuste. La CNN parte de una perdida mucho mas alta (1.5808) y en la ultima epoca sigue en 0.6953 con una perdida de validacion de 0.5872 y una accuracy de validacion de 0.7995: la perdida de validacion deja de mejorar, con lo que hay que revisar la regularizacion. Con el presupuesto ampliado la CNN sigue mejorando pero no llega al F1 macro del baseline denso, de modo que la diferencia no es solo de presupuesto: la arquitectura minima del base (dos bloques convolucionales y GlobalAveragePooling, que resume toda la imagen en 64 caracteristicas) queda por debajo de una capa densa de 784 entradas en este dataset.

Estudio complementario de presupuesto (25 epocas, misma particion y misma semilla). Con ese presupuesto el baseline denso alcanza F1 macro 0.8761 y la CNN 0.8439 (-0.0322), con 37.2 s y 177.4 s de entrenamiento respectivamente. Este experimento es **complementario**: la comparacion oficial del laboratorio sigue siendo la de 8 epocas, y su resultado se reporta sin modificar. Las curvas por epoca se calculan sobre el conjunto de validacion, nunca sobre la prueba.

3. Evidencia visual



Curvas de perdida y accuracy de entrenamiento y validacion de los dos modelos, con la misma particion y la misma semilla. La CNN sigue aprendiendo en la ultima epoca.



Matriz de confusion de la CNN sobre el conjunto de prueba (filas = clase real, columnas = clase predicha). La diagonal concentrate el aciertos; los errores se concentran en las prendas de torso y en el calzado.

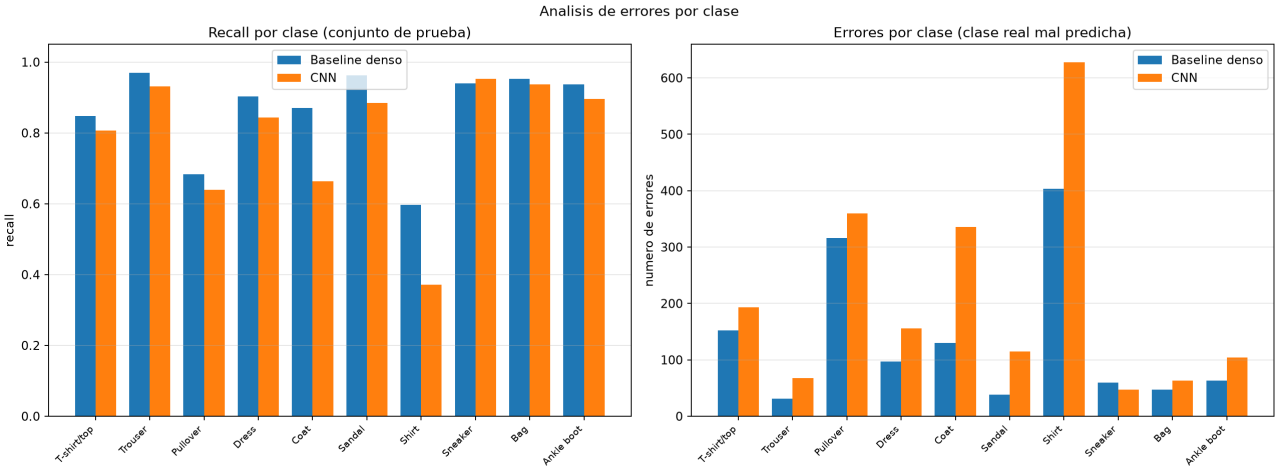
4. Analisis de errores por clase

El promedio global puede esconder una debilidad. La tabla siguiente procede de `reports/cv_errors_by_class.csv`: para cada clase, cuantos ejemplos reales se pierden y cuantos falsos positivos genera.

Clase	Recall denso	Recall CNN	F1 denso	F1 CNN	Errores denso	Errores CNN	Falsos pos. CNN
T-shirt/top	0.848	0.807	0.8185	0.7167	152	193	445
Trouser	0.969	0.932	0.9753	0.9593	31	68	11
Pullover	0.684	0.64	0.7525	0.6716	316	360	266

Dress	0.903	0.844	0.8687	0.7966	97	156	275
Coat	0.87	0.664	0.7824	0.662	130	336	342
Sandal	0.962	0.885	0.9483	0.9166	38	115	46
Shirt	0.597	0.372	0.6525	0.4256	403	628	376
Sneaker	0.94	0.953	0.9344	0.9012	60	47	162
Bag	0.953	0.937	0.9626	0.9268	47	63	85
Ankle boot	0.937	0.896	0.9518	0.9152	63	104	62

Clases que se pierden. Con el presupuesto del manual las tres clases mas afectadas son **Shirt** (recall 0.372, 628 errores), **Pullover** (recall 0.640, 360 errores), **Coat** (recall 0.664, 336 errores) para la CNN y **Shirt** (recall 0.597), **Pullover** (recall 0.684), **T-shirt/top** (recall 0.848) para el baseline denso. El motivo es visual: las prendas de torso (T-shirt/top, Pullover, Coat, Shirt) se distinguen por un cuello y una manga de pocos pixeles, y el calzado (Sandal, Sneaker, Ankle boot) por la suela y el tobillo. Son ambiguedades reales de la imagen de 28x28, no fallos de etiqueta.



Recall por clase y numero de errores por clase de los dos modelos sobre el conjunto de prueba.



Dieciseis ejemplos mal clasificados por la CNN elegidos al azar con semilla fija (42), con la clase real y la prediccion. Los pares visualmente semejantes (Coat/Pullover, Shirt/T-shirt-top, Sandal/Sneaker) confirman que se trata de ambiguedad de la imagen.

5. Decision tecnica y alcance

```
Decision tecnica (calculada sobre las metricas de esta ejecucion)
---------------------------------------------------------------
Modelo entregado: Baseline denso
Presupuesto del manual (8 epocas):
F1 macro denso = 0.8647 | F1 macro CNN = 0.7892 (-0.0755)
Entrenamiento denso = 12.4 s | CNN = 62.4 s (5.0x)
Parametros denso = 50890 | CNN = 19466
Tamano denso = 619.12 KB | CNN = 262.17 KB
Inferencia mediana denso = 0.0379 ms/img | CNN = 0.0899 ms/img (2.4x)
Presupuesto ampliado (25 epocas, experimento complementario):
F1 macro denso = 0.8761 | F1 macro CNN = 0.8439 (-0.0322)
Justificacion: Con 8 epocas el baseline denso alcanza un F1 macro de 0.8647 frente a 0.7892 de la
CNN, y ademas entrena 5.0 veces mas rapido y infiere 2.4 veces mas rapido. La CNN no aporta
mejora predictiva con este presupuesto, de modo que segun el principio Green AI del manual (mas
accuracy no es automaticamente mejor, y hay que justificar el coste) no se justifica su coste de
entrenamiento. Con 25 epocas la CNN sube a 0.8439 y el denso a 0.8761: la diferencia es de
presupuesto y de arquitectura, no de ruido.
Alternativa documentada: La CNN se conserva como alternativa documentada: usa 19466 parametros
frente a 50890 del denso y ocupa 262.17 KB frente a 619.12 KB, es decir, la unica dimension en la
que gana es la huella (memoria y tamano desplegado). Es el modelo que habria que elegir si la
restriccion dominante fuera el tamano del artefacto o la memoria del dispositivo, no el tiempo de
CPU.
Limitacion del benchmark: Fashion-MNIST es didactico; el resultado no se extrapola
a imagenes medicas, OCR ni vigilancia.
Limitacion del entorno: la ejecucion fue en CPU (sin GPU detectada), por lo que los
tiempos de entrenamiento e inferencia son validos para esta maquina y no para un equipo
con GPU.
```

Lectura Green AI. El coste se compara en cinco dimensiones: F1 macro, parametros, tamano en disco, tiempo de entrenamiento y tiempo de inferencia. Con el presupuesto que fija el manual (8 epocas) el baseline denso gana en F1 macro (0.8647 frente a 0.7892), tiempo de entrenamiento (12.4 s frente a 62.4 s), inferencia mediana (0.0379 frente a 0.0899 ms por imagen); la CNN gana unicamente en parametros (19466 frente a 50890), tamano en disco (262.17 frente a 619.12 KB). La CNN es, por tanto, mas compacta pero mas lenta, y en este dataset y con este presupuesto no aporta ninguna mejora predictiva: segun el principio del manual, una mejora que no existe no justifica el coste adicional. La CNN queda documentada como la opcion preferente cuando la restriccion dominante sea la memoria o el tamano del artefacto desplegado, y no el tiempo de CPU.

Alcance y limites. Fashion-MNIST es un benchmark didactico de siluetas de moda en 28x28. Un buen resultado **no** demuestra preparacion para imagenes medicas, OCR, vigilancia ni ningun otro dominio: antes de usar el modelo fuera de esta tarea hay que reentrenarlo y reevaluarlo con datos del dominio. El modelo se entrega con su Model Card, sus metricas por clase y su analisis de errores, y no debe usarse sin supervision humana en decisiones que afecten a personas.

Limitacion del entorno de ejecucion. La ejecucion se hizo en CPU (20 nucleos, GPU detectada: ninguna), con TensorFlow 2.21.0 sobre Python 3.12.15. Los tiempos de entrenamiento e inferencia de este informe son validos para esta maquina y no deben generalizarse a un equipo con GPU.

6. Uso de inteligencia artificial y entrega

Herramienta. Kilo, agente de programacion asistido por IA ejecutado en la terminal del estudiante (modelo MiniMax-M3). **Finalidad:** estructura del proyecto, codigo de los scripts y del modulo de la biblioteca, pruebas, redaccion de la documentacion, ejecucion y depuracion. **La IA no genero datos ni resultados:** todas las cifras proceden de ejecuciones reales del repositorio y estan respaldadas por `reports/*.json` y `reports/*.csv`. **Prompts relevantes:** el encargo que exige que toda cifra salga de los artefactos generados, que el notebook se ejecute de principio a fin sin errores, que la inferencia se mida varias veces con mediana, que la comparacion sea justa con la misma particion y semilla, y que el PDF y el README no puedan contradecir la evidencia. **Verificaciones del estudiante:** particion y semilla comprobadas contra la ejecucion del script, matriz de confusion contrastada con los soportes por clase, parametros y tamano verificados sobre los archivos .keras guardados, curvas leidas epoch a epoch, pruebas y lint en verde antes de publicar y clon limpio ejecutado desde GitHub. **Correcciones realizadas:** se detecto y corrigio la particion relativa del modelo cuando la carpeta de destino estaba fuera del proyecto, se ajusto el diseno de las figuras para que los titulos no se solaparan y se

detecto que la CNN queda en subajuste con el presupuesto del manual, lo que motivo el estudio complementario de presupuesto. El detalle completo esta en `AI_USE_DECLARATION.md`.

Entregable tecnico. `notebooks/01_ejercicio04_cnn_model_card.ipynb` (ejecutado), `src/inf8239_u02_cv/`, `scripts/` (entrenamiento, estudio de presupuesto, PDF y pipeline), `tests/`, `reports/`, `models/best_cnn.keras`, `MODEL_CARD.md`, `AI_USE_DECLARATION.md`, `CAMBIOS_AL_BASE.md` y `README.md`.

Comprobacion	Resultado
Pruebas automaticas	uv run pytest -q : todas en verde
Linter	uv run ruff check . : sin incidencias
Notebook	notebooks/01_ejercicio04_cnn_model_card.ipynb ejecutado, 0 celdas con error
Clon limpio	bash scripts/run_all.sh ejecutado desde un clon de GitHub sin errores

Repositorio: https://github.com/edvvdev/unidad02-02